MERN STACK

TRAINING TR-103 REPORT DAY 22

20 July 2026

Overview:

The twenty-second day of the MERN Stack training focused on handling POST requests in Express.js and performing database operations using MongoDB and Mongoose.I learned how to receive data from the client using the request object (`req`), send responses using the response object (`res`), and perform asynchronous database operations with async and await. The session also covered Mongoose Models, findOne(), and save() methods to validate and store user data in MongoDB.

Learning Objectives:

- Understand the POST method in Express.js.
- Learn about the request (`req`) and response (`res`) objects.
- Understand asynchronous programming using `async` and `await`.
- Learn how Mongoose Models interact with MongoDB.
- Use `findOne()` to search documents.
- Use `save()` to insert data into MongoDB.

MongoDB

POST Method

The POST method is an HTTP request method used to send data from the client to the server. It is mainly used to create new records in a database.

Features:

- Sends data from client to server.
- Used for creating new records.
- Commonly used in registration and login forms.
- Handles data securely through the request body.

Syntax:

```
app.post(path, callback_function)
```

Request Object (req)

The request object (req) contains the data sent by the client to the server. User input can be accessed using req.body.

Features:

- Stores client request data.
- Accesses form or JSON data.
- Used to process user input.

Syntax:

```
req.body.property_name
```

Response Object (res)

The response object (res) is used to send data or messages back to the client after processing the request.

Features:

- Sends API responses.
- Returns success or error messages.
- Sends JSON data to the frontend.

Syntax:

```
res.send(data)
```

Async and Await

async and await are JavaScript features used to handle asynchronous operations. They are commonly used with database queries to wait until an operation is completed.

Features:

- Simplifies asynchronous programming.
- Waits for database operations to complete.
- Improves code readability.
- Works with Promises.

Syntax:

```
async function functionName(){

await asynchronous_operation();

}
```

Mongoose Model

A Mongoose Model is an object created from a schema that provides methods to interact with a MongoDB collection.

Features:

- Connects schema with collections.
- Performs CRUD operations.
- Validates data before storing.
- Simplifies database interaction.

Syntax:

```
mongoose.model(collectionName, schema)
```

findOne() Function

The findOne() method searches for a single document in a MongoDB collection that matches the specified condition.

Features:

- Retrieves one matching document.
- Commonly used to check existing users or records.
- Returns null if no document is found.

Syntax:

```
Model.findOne(condition)
```

save() Function

The save() method stores a new document in a MongoDB collection.

Features:

- Inserts new data into the database.
- Saves documents created using Mongoose models.
- Returns the saved document after successful insertion.

Syntax:

```
document.save()
```

Status Code Response

The server sends custom JSON responses to indicate whether an operation was successful or not.

Common Responses:

- statuscode: 1 – Data inserted successfully.
- statuscode: 2 – Record already exists (e.g., duplicate email).
- statuscode: 0 – An error occurred during the operation.

Syntax:

```
res.send({
    statuscode: 1,
    msg: "Success"
});
```

Task –1

```
const mongoose=require("mongoose")
const express=require("express")
const cors=require("cors")
const app=express()
app.use(cors())
app.use(express.json())
app.listen(9000,()=>{
    console.log("connected")
});
mongoose.connect("mongodb://127.0.0.1:27017/newcrud").then(()=>{
    console.log("DB CONNECTED")
}).catch((err)=>{
    console.log(err);
})

const schema=mongoose.Schema({
    Name:String,
    Email:String,
    Age:Number
})

const usermodel=mongoose.model("Detail",schema,"Detail")

app.post("/api/postuser",async(req,res)=>{
const findser=await usermodel.findOne({Email:req.body.mail});
if(findser){
    res.send({statuscode:2,msg:"Mail aleready exist"})
}else{

    const data=await usermodel({
        Name:req.body.name,
        Email:req.body.mail,
        Age:req.body.age
    });
    const savedata=await data.save();
    if(savedata){
        res.send({statuscode:1,msg:"Success"})
    }else{
        res.send({statuscode:0,msg:"error"})
    }
}
})
```

```jsx
import { useState } from "react";

export const Postdata = () => {
  const [name, setname] = useState("");
  const [age, setage] = useState("");
  const [mail, setmail] = useState("");

  const sub = async () => {
    try {
      const data = await fetch("http://localhost:9000/api/posture", {
        method: "POST",
        headers: {
          "Content-Type": "application/json",
        },
        body: JSON.stringify({ name, mail, age }),
      });

      if (data.ok) {
        const res = await data.json();

        if (res.statuscode === 1) {
          alert(res.msg);
        } else {
          alert(res.msg);
        }
      } else {
        alert("Request failed!");
      }
    } catch (err) {
      console.error(err);
      alert("Server is not running!");
    }
  };

  return (
    <>
      <input
        type="text"
        placeholder="Enter name"
        value={name}
        onChange={(e) => setname(e.target.value)}
      />

      <input
        type="number"
        placeholder="Enter age"
        value={age}
        onChange={(e) => setage(e.target.value)}
      />

      <input
        type="email"
        placeholder="Enter email"
        value={mail}
        onChange={(e) => setmail(e.target.value)}
      />

      <button onClick={sub}>Click</button>
    </>
  );
};
```

Output:



```
_id: ObjectId('6a5da4ba2aa9c6625933997a')
Name : "jeeya"
Email : "jeeyathapar786@gmail.com"
Age : 21
__v : 0
```


Conclusion:

By the end of Day 22, I gained a clear understanding of handling POST requests in Express.js and performing database operations using MongoDB and Mongoose. I learned how to receive user data, validate existing records using `findOne()`, insert new documents using `save()`, and send appropriate responses back to the client, strengthening my backend development skills.